EXP 7 COMPUTERNETWORK

Flow Control at Data Link Layer Using Sliding Window Protocol

Aim

To implement the Sliding Window Protocol for flow control at the Data Link Layer and simulate the transmission of frames between two nodes.

Procedure

1. Open a suitable programming environment and write a program to simulate the Sliding Window Protocol.
2. Initialize the sender and receiver with a set of data frames to be transmitted. Specify the window size, which determines the maximum number of frames that can be sent without waiting for acknowledgment.
3. The sender transmits frames up to the specified window size to the receiver.
4. The receiver accepts the incoming frames and sends acknowledgments to the sender for successfully received frames.
5. After receiving acknowledgments, the sender slides its window forward and transmits additional frames.
6. Continue the transmission and acknowledgment process until all frames have been sent and

received.

7. Simulate a situation in which a frame or acknowledgment is lost. Observe how the sender responds according to the implemented protocol.
8. If a timeout occurs, retransmit the required frame or frames, depending on the selected Sliding Window Protocol.
9. Display the transmitted frames, received acknowledgments, window movement, and retransmissions, if any.
10. Repeat the experiment with different window sizes and observe how the window size affects the transmission process.

Sender.c:

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>

#ifdef _WIN32
#include <windows.h>
void delay(int seconds)
{
   Sleep(seconds * 1000);
}
#else
#include <unistd.h>
void delay(int seconds)
{
```

```
    sleep(seconds);
}
#endif

#define SENDER_FILE "Sender_Buffer.txt"
#define RECEIVER_FILE "Receiver_Buffer.txt"

void clearFile(const char *filename)
{
    FILE *fp = fopen(filename, "w");

    if (fp != NULL)
        fclose(fp);
```

```
}

int readAcknowledgement(int *type, int *number)
{
   FILE *fp;
   char line[100];

   fp = fopen(RECEIVER_FILE, "r");

   if (fp == NULL)
      return 0;

   if (fgets(line, sizeof(line), fp) == NULL)
   {
      fclose(fp);
      return 0;
   }

   fclose(fp);

   if (sscanf(line, "ACK:%d", number) == 1)
   {
      *type = 1;
      return 1;
   }

   if (sscanf(line, "NACK:%d", number) == 1)
   {
      *type = 2;
      return 1;
   }
```

```
    return 0;
}

void writeFrames(char *message, int start, int end)
{
    FILE *fp;

    fp = fopen(SENDER_FILE, "w");

    if (fp == NULL)
    {
        printf("Error: Cannot open Sender_Buffer.txt\n");
        exit(1);
    }

    printf("\n--------------------------\n");
    printf("Sending frames %d to %d\n", start, end);
    printf("--------------------------\n");

    for (int i = start; i <= end; i++)
    {
        fprintf(fp, "Frame:%d DATA:%c\n", i, message[i]);

        printf("Frame No: %d   DATA: %c\n",
               i,
               message[i]);
    }

    fclose(fp);
}
```

```
int main()
{
    char message[500];
    int windowSize;

    int totalFrames;
    int base = 0;

    int ackType;
    int ackNumber;

    printf("========== SLIDING WINDOW SENDER ==========\n\n");

    printf("Enter window size: ");
    scanf("%d", &windowSize);

    if (windowSize <= 0)
    {
        printf("Invalid window size.\n");
        return 1;
    }

    getchar();

    printf("Enter text message: ");
    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    totalFrames = strlen(message);
```

```
if (totalFrames == 0)
{
   printf("Message cannot be empty.\n");
   return 1;
}

clearFile(SENDER_FILE);
clearFile(RECEIVER_FILE);

printf("\nMessage: %s\n", message);
printf("Total frames: %d\n", totalFrames);
printf("Window size: %d\n", windowSize);

while (base < totalFrames)
{
   int end = base + windowSize - 1;

   if (end >= totalFrames)
      end = totalFrames - 1;

   writeFrames(message, base, end);

   printf("\nSender_Buffer.txt updated.");
   printf("\nWaiting for acknowledgement");

   while (1)
   {
      delay(2);

      if (readAcknowledgement(&ackType, &ackNumber))
```

```
    {
        if (ackType == 1)
        {
            printf("\nReceived ACK:%d\n", ackNumber);

            if (ackNumber >= base)
            {
                base = ackNumber + 1;

                clearFile(RECEIVER_FILE);

                printf("Sliding window moved.\n");
                break;
            }
        }
        else if (ackType == 2)
        {
            printf("\nReceived NACK:%d\n", ackNumber);

            printf("Resending frames from frame %d...\n",
                  ackNumber);

            clearFile(RECEIVER_FILE);

            if (ackNumber >= base)
                base = ackNumber;

            break;
        }
    }
}
```

```
    }

    clearFile(SENDER_FILE);

    printf("\n=======================================\n");
    printf("All frames transmitted successfully!\n");
    printf("Message transmission completed.\n");
    printf("=======================================\n");

    return 0;
}
```

Receiver.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#ifdef _WIN32
#include <windows.h>
void delay(int seconds)
{
    Sleep(seconds * 1000);
}
#else
#include <unistd.h>
void delay(int seconds)
{
    sleep(seconds);
}
```

```
#endif

#define SENDER_FILE "Sender_Buffer.txt"
#define RECEIVER_FILE "Receiver_Buffer.txt"

int readFrames(int *frameNumbers, char *data, int maxFrames)
{
   FILE *fp;
   char line[200];

   int count = 0;

   fp = fopen(SENDER_FILE, "r");

   if (fp == NULL)
      return 0;

   while (fgets(line, sizeof(line), fp) != NULL &&
         count < maxFrames)
   {
      int frameNo;
      char ch;

      if (sscanf(line, "Frame:%d DATA:%c",
               &frameNo, &ch) == 2)
      {
         frameNumbers[count] = frameNo;
         data[count] = ch;

         count++;
      }
```

```
    }

    fclose(fp);

    return count;
}

void writeACK(int number)
{
    FILE *fp = fopen(RECEIVER_FILE, "w");

    if (fp == NULL)
        return;

    fprintf(fp, "ACK:%d\n", number);

    fclose(fp);
}

void writeNACK(int number)
{
    FILE *fp = fopen(RECEIVER_FILE, "w");

    if (fp == NULL)
        return;

    fprintf(fp, "NACK:%d\n", number);

    fclose(fp);
}
```

```
int main()
{
    int expectedFrame = 0;

    int frameNumbers[100];
    char data[100];

    int lastFirstFrame = -1;

    printf("========== SLIDING WINDOW RECEIVER ==========\n\n");

    printf("Receiver is waiting for frames...\n");

    while (1)
    {
        int count = readFrames(frameNumbers, data, 100);

        if (count > 0)
        {
            /*
             * Process only a new Sender_Buffer window.
             */
            if (frameNumbers[0] != lastFirstFrame)
            {
                int valid = 1;

                printf("\n--------------------------\n");
                printf("Frames received:\n");
                printf("--------------------------\n");

                for (int i = 0; i < count; i++)
```

```
{
   printf("Frame No: %d   DATA: %c\n",
         frameNumbers[i],
         data[i]);

   if (frameNumbers[i] != expectedFrame)
   {
      printf("\nExpected Frame: %d\n",
            expectedFrame);

      printf("Received Frame: %d\n",
            frameNumbers[i]);

      printf("ERROR DETECTED!\n");

      printf("Sending NACK:%d\n",
            expectedFrame);

      writeNACK(expectedFrame);

      valid = 0;
      break;
   }

   expectedFrame++;
}

if (valid)
{
   int ackNumber = expectedFrame - 1;
```

```
                printf("\nAll frames received correctly.\n");

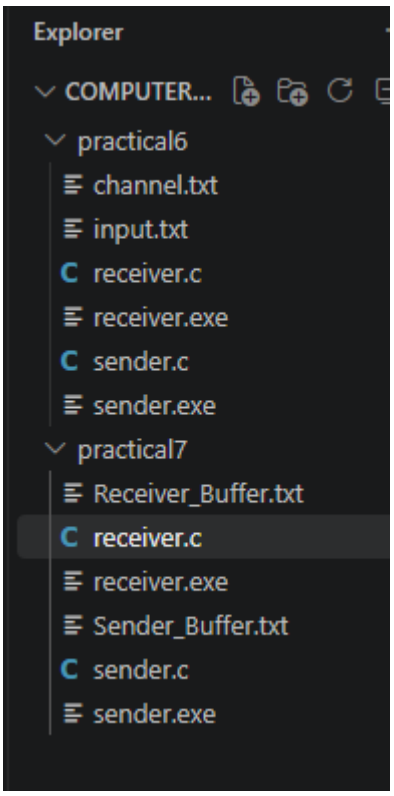
                printf("Sending ACK:%d\n",
                    ackNumber);

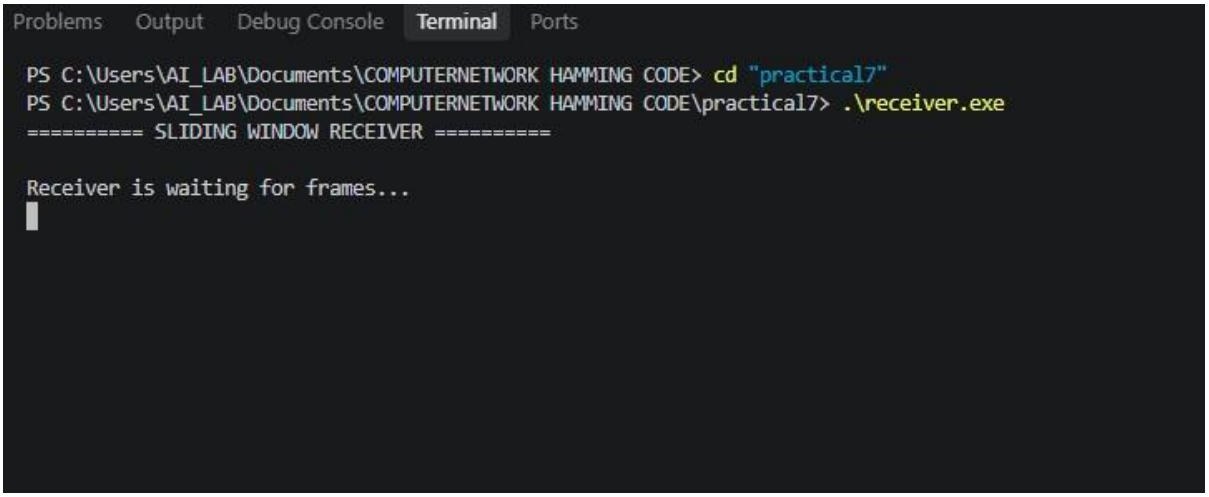
                writeACK(ackNumber);
            }

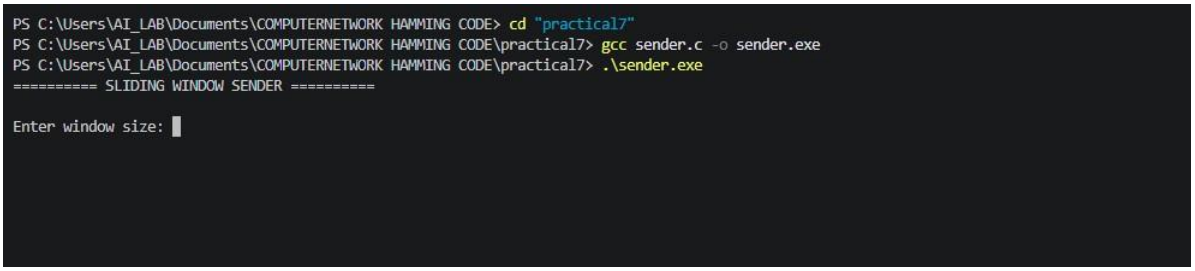
            lastFirstFrame = frameNumbers[0];
        }
    }

    delay(1);
  }

  return 0;
}
```




Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE> cd "practical7"
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE\practical7> .\receiver.exe
========== SLIDING WINDOW RECEIVER ==========
Receiver is waiting for frames...


PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE> cd "practical7"
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE\practical7> gcc sender.c -o sender.exe
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE\practical7> .\sender.exe
========== SLIDING WINDOW SENDER ==========
Enter window size:

Sender:

```
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE> cd "
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE\pract
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE\pract
========== SLIDING WINDOW SENDER ==========

Enter window size: 3
Enter text message: computer

Message: computer
Total frames: 8
Window size: 3

--------------------------------------
Sending frames 0 to 2
--------------------------------------
Frame No: 0   DATA: c
Frame No: 1   DATA: o
Frame No: 2   DATA: m

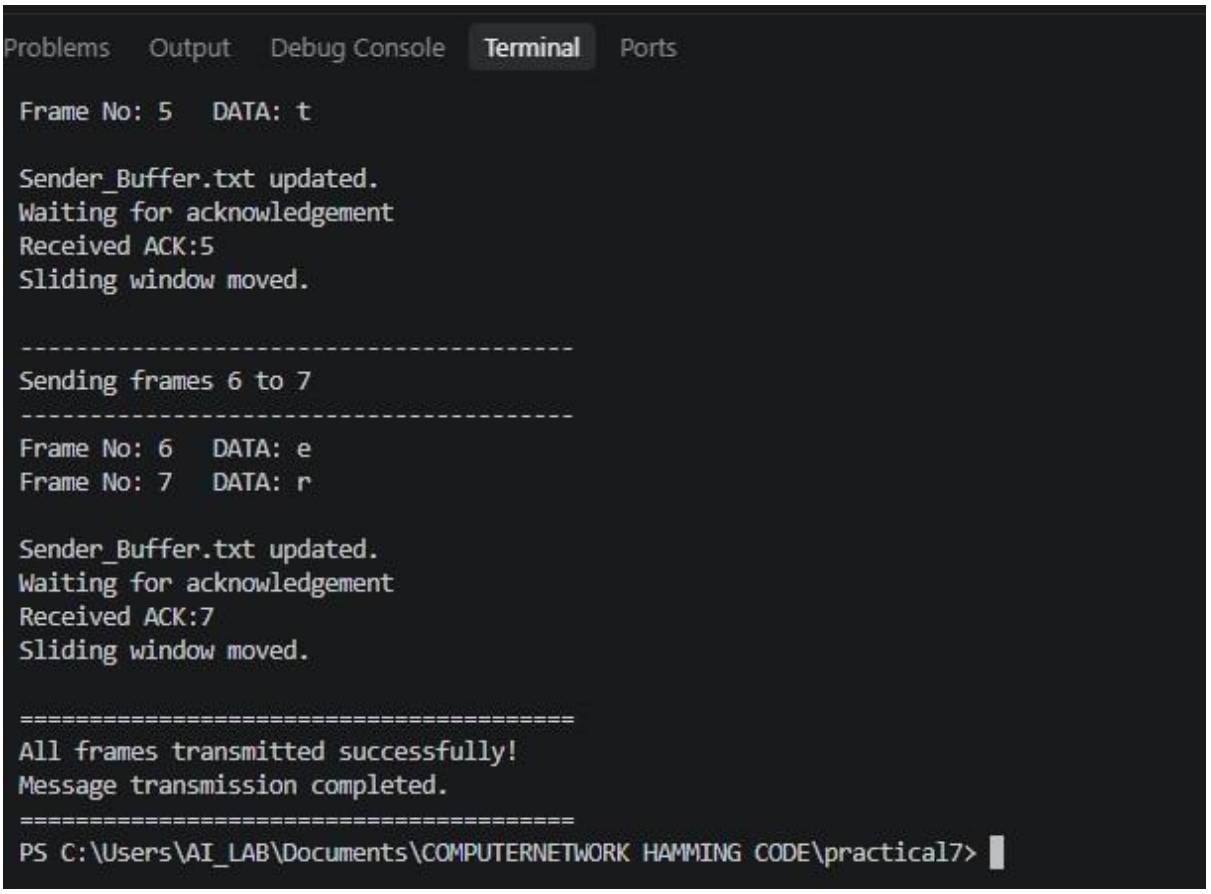
Sender_Buffer.txt updated.
Waiting for acknowledgement
Received ACK:2
Sliding window moved.
```

```
--------------------------------------
Sending frames 3 to 5
--------------------------------------
Frame No: 3   DATA: p
Frame No: 4   DATA: u
Frame No: 5   DATA: t

Sender_Buffer.txt updated.
Waiting for acknowledgement
Received ACK:5
Sliding window moved.

--------------------------------------
Sending frames 6 to 7
--------------------------------------
Frame No: 6   DATA: e
Frame No: 7   DATA: r

Sender_Buffer.txt updated.
Waiting for acknowledgement
Received ACK:7
Sliding window moved.
```


Problems
Output
Debug Console
Terminal
Ports
Frame No: 5 DATA: t
Sender_Buffer.txt updated.
Waiting for acknowledgement
Received ACK:5
Sliding window moved.
Sending frames 6 to 7
Frame No: 6 DATA: e
Frame No: 7 DATA: r
Sender_Buffer.txt updated.
Waiting for acknowledgement
Received ACK:7
Sliding window moved.
All frames transmitted successfully!
Message transmission completed.
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE\practical7>

Receiver:

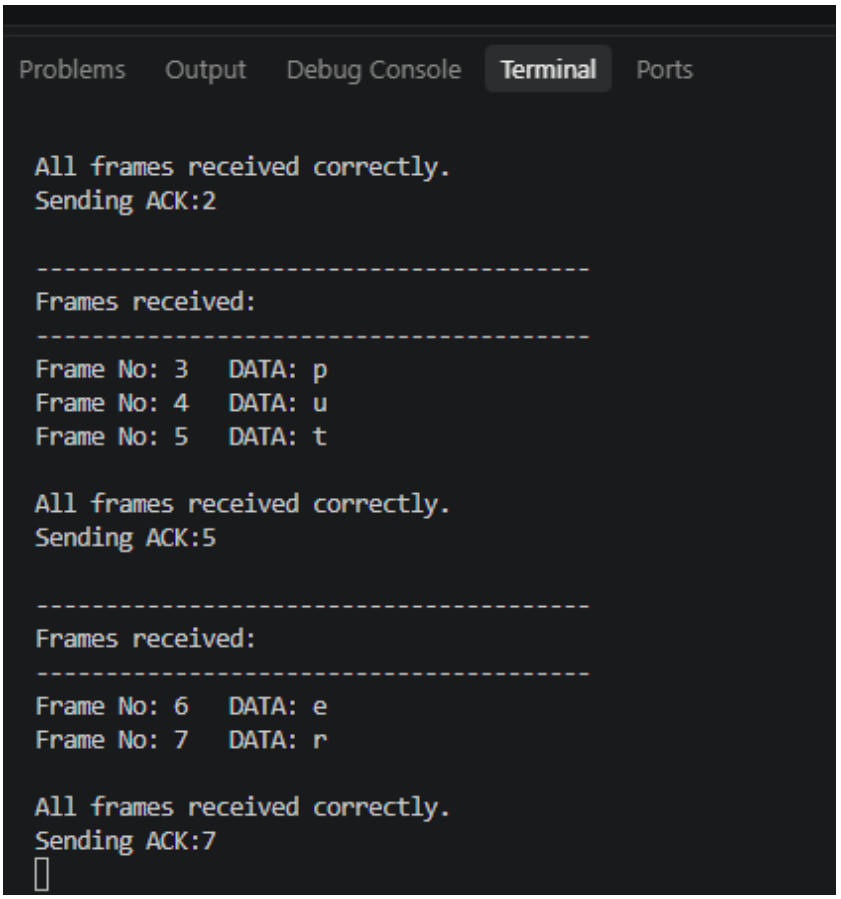
```
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE> cd "practical7"
PS C:\Users\AI_LAB\Documents\COMPUTERNETWORK HAMMING CODE\practical7> .\receiver.exe
========== SLIDING WINDOW RECEIVER ==========

Receiver is waiting for frames...

---------------------------------------
Frames received:
---------------------------------------
Frame No: 0   DATA: c
Frame No: 1   DATA: o
Frame No: 2   DATA: m

All frames received correctly.
Sending ACK:2

---------------------------------------
Frames received:
---------------------------------------
Frame No: 3   DATA: p
Frame No: 4   DATA: u
Frame No: 5   DATA: t
```



Result

The Sliding Window Protocol was implemented successfully. Frame transmission, acknowledgment handling, and window movement between the sender and receiver were simulated, demonstrating flow control at the Data Link Layer.